SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**

Question Count: 5

Code of Conduct

I Ch.Tharunika Reddy (192324307) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student:

Ch.Tharunika Reddy

Assessment Tasks Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy.

Design a solution using cloud-native backup, replication, and versioning.
Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved. Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured. Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

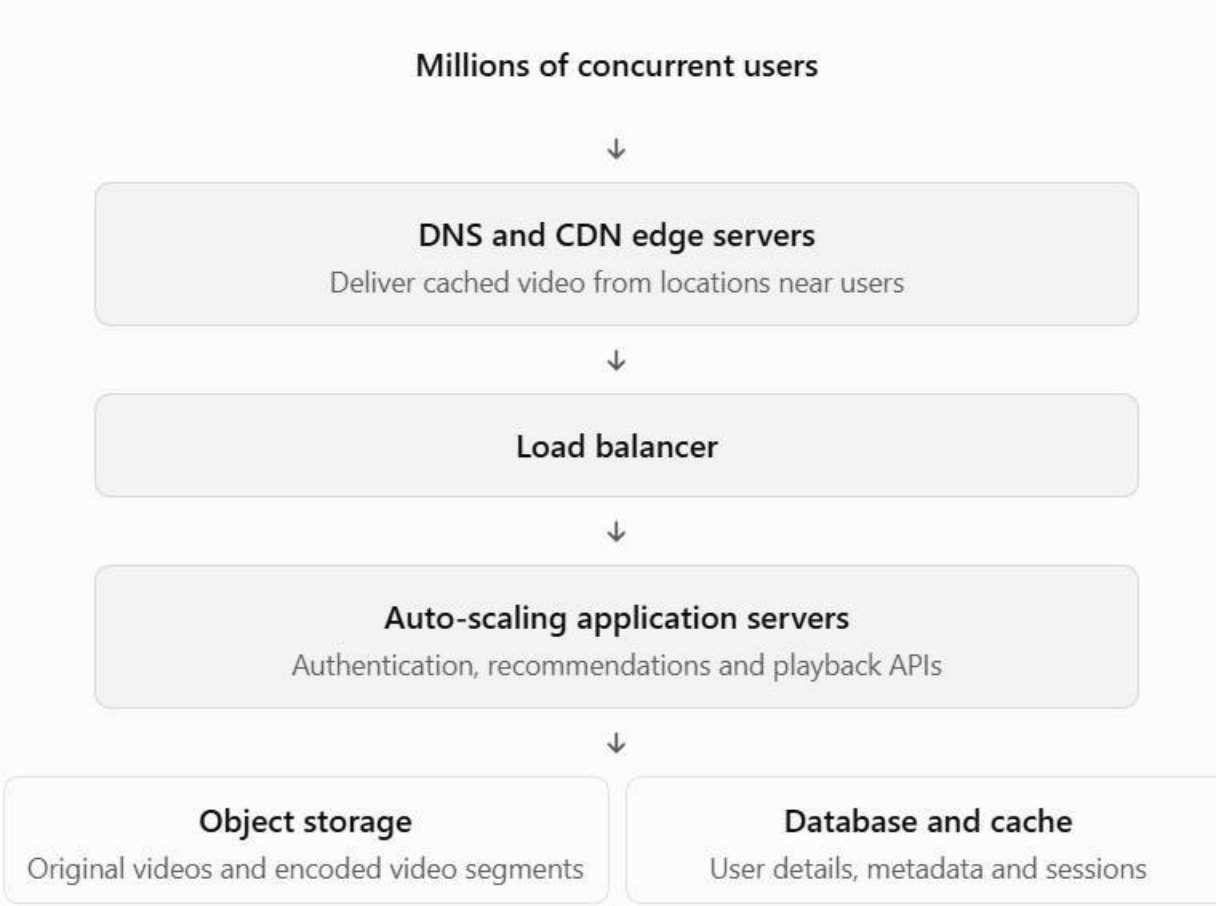
Solutions:

1. Cloud Architecture for a Video Streaming Platform

Scenario: A media company needs to support millions of concurrent users with minimal buffering.

1.1 Proposed architecture

The company should use a scalable cloud architecture with a Content Delivery Network (CDN), load balancers, auto-scaling, and cloud storage.



1.2 Scalability and caching techniques

- **Horizontal scaling:** Add more application server instances to handle increasing requests.
- **CDN caching:** Store frequently requested video segments at edge locations near users.
- **Object storage:** Store videos in durable, scalable cloud storage.
- **Adaptive bitrate streaming:** Adjust video quality according to the user's available bandwidth.
- **Database caching:** Use an in-memory cache for frequently accessed metadata and session information.

1.3 Role of load balancing and auto-scaling

Load balancing distributes incoming requests across healthy servers, preventing one server from becoming overloaded.

Auto-scaling automatically increases server instances during peak demand and reduces them during low demand. This maintains performance while avoiding unnecessary resource usage.

1.4 Cost optimization strategies

1. Use CDN caching to reduce repeated requests to origin servers.
2. Use reserved or committed-use capacity for predictable workloads.
3. Use spot instances for suitable background processing tasks.
4. Automatically scale down unused resources.
5. Use storage lifecycle policies to move older videos to cheaper storage tiers.
6. Monitor bandwidth, storage, and compute costs.

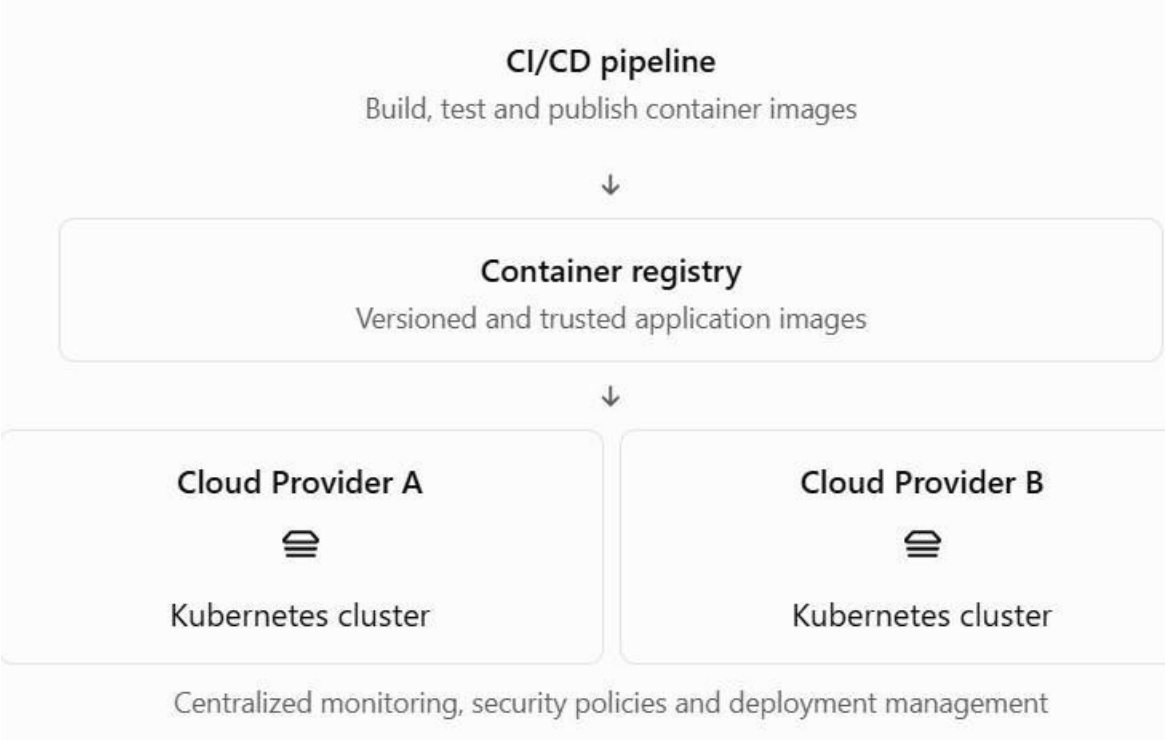
Conclusion: A CDN combined with load balancing, auto-scaling, adaptive streaming, and cloud storage provides smooth playback, high scalability, and cost efficiency.

2. Multi-Cloud Container Deployment for a Fintech Startup

Scenario: A fintech startup deploys containerized applications across multiple cloud providers but faces deployment consistency and management problems.

2.1 Proposed solution

Use Docker containers and Kubernetes orchestration to deploy and manage applications consistently across cloud platforms.



2.2 Container orchestration

Kubernetes manages containers across multiple machines and provides:

- **Automated deployment:** Deploys the same application image across environments.
- **Self-healing:** Restarts failed containers and replaces unhealthy instances.
- **Rolling updates:** Releases new versions gradually without stopping the entire application.
- **Resource management:** Allocates CPU and memory efficiently.
- **Horizontal scaling:** Adds or removes application replicas based on workload.

2.3 Maintaining consistency across clouds

1. Package applications using Docker.

2. Store versioned images in a secure container registry.
3. Use Kubernetes manifests or Helm charts to define deployments.
4. Use Infrastructure as Code (IaC) to create consistent cloud infrastructure.
5. Automate testing and deployments through a CI/CD pipeline.
6. Keep secrets and configuration outside the container image.

2.4 Service discovery and scaling

Service discovery: Kubernetes Services provide stable names and network endpoints for application components. DNS allows services to locate each other without hardcoding IP addresses.

Scaling: Kubernetes Horizontal Pod Autoscaler adjusts the number of replicas based on CPU, memory, or custom metrics. Cluster autoscaling can add worker nodes when required.

For communication between clouds, secure DNS, private connectivity or VPNs, and service-mesh or gateway solutions can be used where appropriate.

2.5 Benefits and risks of multi-cloud

Benefits	**Risks**
Reduced dependence on one provider	More complex management
Improved resilience	Higher networking costs
Access to different cloud services	Different provider configurations
Flexible resource selection	Security and compliance challenges
Potential for better availability	Data transfer and synchronization issues

Conclusion: Docker, Kubernetes, CI/CD, Infrastructure as Code, and centralized monitoring improve deployment consistency and scalability. Strong identity controls, encryption, and cost monitoring are essential for a secure multi-cloud environment.

3. Backup and Disaster Recovery for an E-Learning Platform

Scenario: An e-learning platform loses data due to accidental deletion and requires a robust backup and disaster recovery strategy.

3.1 Proposed architecture

The platform should use automated backups, data replication, versioning, and a separate disaster recovery environment.



3.2 Backup, replication, and versioning

- **Cloud-native backup:** Schedule regular backups of databases, application configurations, and important files.
- **Data replication:** Copy data to another availability zone or region to improve resilience against infrastructure failures.
- **Versioning:** Enable object versioning so accidentally deleted or overwritten learning materials can be restored.
- **Immutable backups:** Protect backup copies from accidental deletion or ransomware by preventing modification for a defined retention period.
- **Access control:** Restrict backup deletion permissions to authorized administrators.

Replication alone is not sufficient because accidental deletion may also replicate. Therefore, independent backups and version history are necessary.

3.3 RTO and RPO

- **Recovery Time Objective (RTO):** The maximum acceptable time taken to restore the service after a failure.
- **Recovery Point Objective (RPO):** The maximum acceptable amount of data loss measured in time.

For example, if the RTO is 1 hour, the platform should restore service within 1 hour. If the RPO is 15 minutes, the recovery process should limit data loss to at most 15 minutes of changes.

Objective	How it is achieved
RTO	Automated recovery, standby infrastructure, tested restoration procedures
RPO	Frequent backups, transaction logs and continuous or frequent replication

These targets must be defined according to business requirements; the examples above are illustrative.

3.4 Business continuity steps

1. Identify critical applications and databases.
2. Configure automatic backups and cross-region copies.
3. Enable versioning and retention policies.
4. Monitor backup completion and replication status.
5. Detect failures and declare a disaster recovery event.
6. Restore data or promote the standby environment. 7. Verify data integrity and application functionality.
8. Redirect users to the recovered service.
9. Test the disaster recovery plan periodically and update it.

Conclusion: Automated backups, versioning, replication, and tested recovery procedures protect the platform from accidental deletion and infrastructure failures while supporting business continuity.

4. Secure Hybrid Cloud Architecture for a Government Agency

Scenario: A government agency wants to store sensitive data on-premises while using public cloud resources for non-sensitive applications.

4.1 Proposed architecture

A hybrid cloud combines private on-premises infrastructure with public cloud services. Sensitive information remains in the approved on-premises environment, while suitable public-facing applications and non-sensitive workloads use the public cloud.



4.2 Secure communication between environments

1. Establish a site-to-site VPN or dedicated private network connection.
2. Encrypt data in transit using secure protocols such as TLS.
3. Configure firewalls to allow only approved traffic.
4. Separate sensitive systems using network segmentation.
5. Use private endpoints where available to reduce exposure to the public internet.
6. Monitor traffic and maintain audit logs.

4.3 Identity management and encryption

Identity and Access Management (IAM):

- Use centralized identity management and single sign-on where appropriate.
- Apply role-based access control (RBAC).
- Follow the principle of least privilege.
- Enable multi-factor authentication for privileged and sensitive access.
- Regularly review user accounts and permissions.

Encryption:

- Encrypt sensitive data at rest using approved encryption mechanisms.
- Encrypt data in transit using TLS or an equivalent secure protocol.
- Use controlled key management and rotate keys according to policy.
- Protect encryption keys separately from the data they secure.

The agency must also follow applicable government regulations, data residency requirements, and security policies.

4.4 Challenges in hybrid cloud

Challenge	Solution
Complex management	Centralized monitoring and management tools
Network latency	Optimize connectivity and minimize unnecessary data transfers
Security risks	IAM, encryption, firewalls and regular audits
Integration issues	Standard APIs and compatible systems
Compliance requirements	Data classification and policy enforcement
Operational costs	Resource monitoring and cost planning

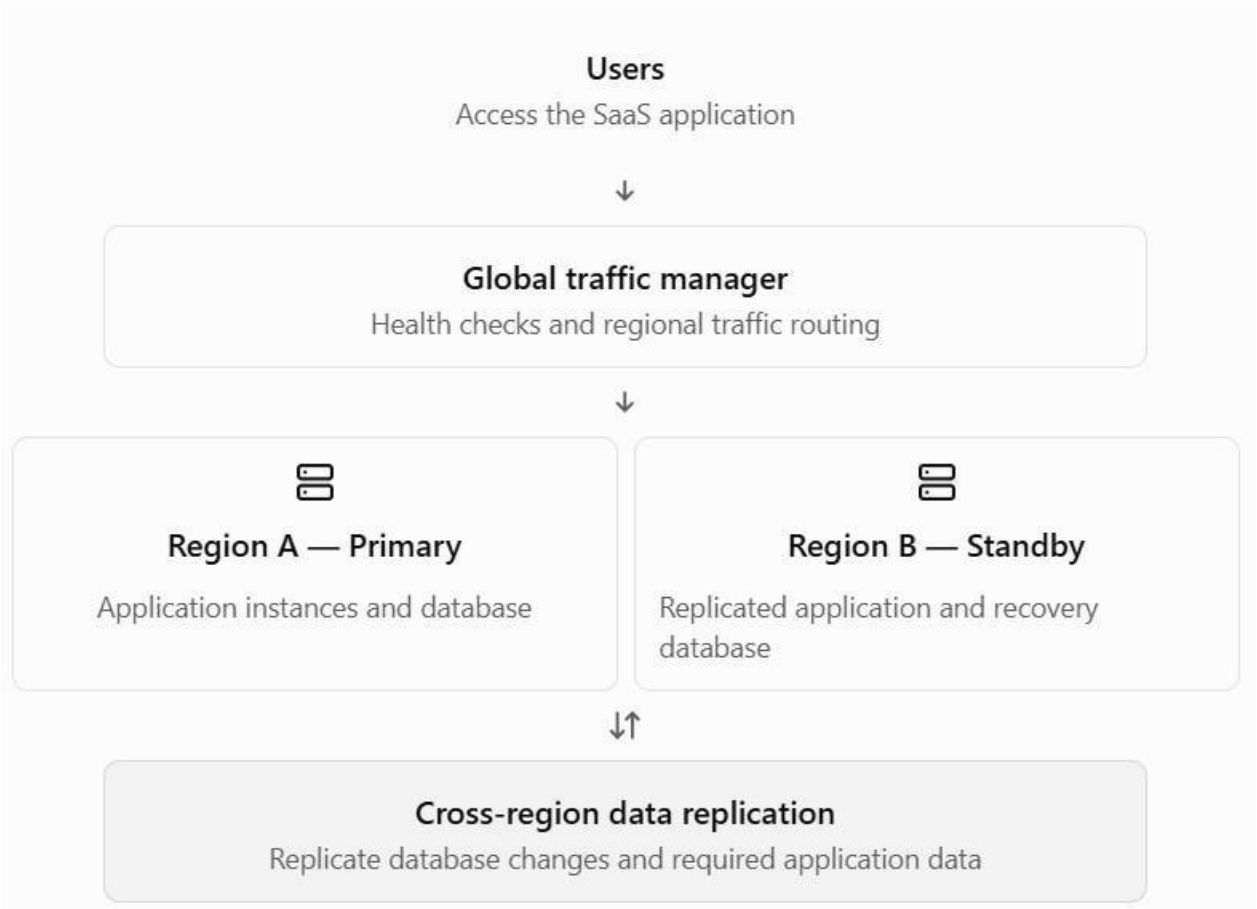
Conclusion: A hybrid cloud provides flexibility while retaining control over sensitive information. Secure connectivity, strong identity management, encryption, and compliance monitoring are essential to protect government data.

5 Multi-Region Deployment for High Availability and Fault Tolerance

Scenario: A SaaS provider needs its application to continue operating even if one cloud region becomes unavailable.

5.1 Proposed architecture

Deploy the application across at least two geographically separate cloud regions. Use health checks, replicated data, and automated or controlled failover to maintain service availability.



5.2 Multi-region deployment strategy

1. Deploy the same application version in multiple regions.
2. Configure a global load balancer or traffic manager.
3. Set up health checks to detect application and regional failures.
4. Maintain sufficient capacity in the secondary region.
5. Replicate required databases and application data across regions.
6. Test regional failover and failback procedures.

7. Keep configurations, security policies, and software versions consistent.

A warm standby keeps a reduced-capacity secondary environment ready to scale up. An active-active setup serves traffic from multiple regions simultaneously, improving utilization but requiring more complex data and traffic management.

5.3 Data replication and failover

Data replication:

- Use synchronous replication when supported and appropriate for tightly controlled latency and consistency requirements.
- Use asynchronous replication when cross-region latency makes synchronous replication impractical.
- Monitor replication lag and data consistency.
- Back up data independently because replication can also copy accidental changes or deletions.

Failover:

1. Continuously monitor regional health.
2. Detect an outage using health checks and alerts.
3. Determine whether the failure requires regional failover.
4. Promote the secondary database if required, ensuring the old primary cannot continue accepting conflicting writes.
5. Redirect traffic to the healthy region.
6. Validate application functionality and data consistency.
7. Restore the failed region and perform a controlled failback when safe.

5.4 Monitoring and alerting

Monitor the following metrics:

- Application availability and response time.
- HTTP error rates and failed requests.
- CPU, memory, and network utilization.
- Database health and replication lag.
- Failed health checks and regional outages.
- Backup status and recovery readiness.

Configure alerts through a centralized monitoring platform. Send critical alerts to the operations team through approved notification channels. Maintain dashboards, logs, incident procedures, and regular disaster recovery tests.

5.5 Benefits and challenges

Benefits	Challenges
High availability	Higher infrastructure costs
Improved fault tolerance	Data consistency issues
Better disaster recovery	Complex deployment management
Geographic redundancy	Replication latency
Potentially lower user latency	Failover and failback complexity

Conclusion: A multi-region deployment with global traffic management, replicated data, health monitoring, and tested failover procedures helps a SaaS provider maintain availability during regional failures.